

C# Skripts & Erklärungen

PlayerMovement

```
using UnityEngine;
```

```
public class PlayerMovement : MonoBehaviour
{
    public float moveSpeed = 5f; // Bewegungsgeschwindigkeit
    public float jumpForce = 7f; // Sprungkraft
    private Rigidbody2D rb; // Rigidbody für den Spieler
    private bool isGrounded = true; // Überprüft, ob der Spieler den Boden berührt

    void Start()
    {
        rb = GetComponent<Rigidbody2D>(); // Holen des Rigidbody
    }

    void Update()
    {
        // Bewegung nach links/rechts
        float move = Input.GetAxis("Horizontal");
        transform.Translate(new Vector2(move * moveSpeed * Time.deltaTime, 0));

        // Springen, wenn die W-Taste oder Pfeiltaste nach oben gedrückt wird und der Spieler auf
        // dem Boden ist
        if ((Input.GetKeyDown(KeyCode.W) || Input.GetKeyDown(KeyCode.UpArrow)) &&
            isGrounded)
        {
            rb.AddForce(Vector2.up * jumpForce, ForceMode2D.Impulse); // Sprung auslösen
            isGrounded = false; // Spieler ist jetzt in der Luft
        }
    }

    // Wenn der Spieler den Boden berührt, kann er wieder springen
    void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Ground"))
        {
            isGrounded = true; // Spieler ist wieder auf dem Boden
        }
    }
}
```

PlayerKick

```
using UnityEngine;
public class PlayerKick : MonoBehaviour
{
    public float kickForce;    // Wie stark Messi den Ball tritt
    private Rigidbody2D ballRb;    // Speichert das Rigidbody vom Ball
    private bool ballInRange = false; // Ist der Ball gerade in Reichweite?
    private Animator animator;    // Für Animation
    public AudioSource kickSound;    // Sound, der beim Tritt abgespielt wird
    void Start()
    {
        animator = GetComponentInChildren<Animator>(); // Animator holen
    }

    void Update()
    {
        // Wenn SPACE gedrückt wird und Ball in Reichweite ist
        if (Input.GetKeyDown(KeyCode.Space) && ballInRange && ballRb != null)
        {
            // Richtung vom Spieler zum Ball
            Vector2 direction = (ballRb.position - (Vector2)transform.position).normalized;

            // Ball wird mit Kraft weggeschossen
            ballRb.AddForce(direction * kickForce, ForceMode2D.Impulse);

            // Animation auslösen
            animator.SetTrigger("KickTrigger");

            // Sound abspielen
            kickSound.Play();
        }
    }

    // Wenn der Spieler den Ball berührt
    void OnCollisionEnter2D(Collision2D collision)
    {
        // Wenn Ball berührt wird → merken
        if (collision.gameObject.CompareTag("Ball"))
        {
            ballRb = collision.gameObject.GetComponent<Rigidbody2D>(); // Rigidbody holen
            ballInRange = true; // Ball ist in Reichweite
        }
    }

    // Wenn Spieler den Ball wieder verlässt
    void OnCollisionExit2D(Collision2D collision)
    {
        // Wenn Ball nicht mehr berührt wird
        if (collision.gameObject.CompareTag("Ball"))
        {
            ballInRange = false;
        }
    }
}
```

```

        ballInRange = false;
        ballRb = null;
    }
}

using UnityEngine;

public class BallSpin : MonoBehaviour
{
    public float spinSpeed; // Wie stark sich der Ball drehen soll
    private Rigidbody2D rb; // Rigidbody des Balls
    public AudioSource latteAudio; // Latte-Sound

    void Start()
    {
        rb = GetComponent<Rigidbody2D>(); // Holt das Rigidbody vom Ball
    }

    void FixedUpdate()
    {
        // Berechnet Drehkraft basierend auf der Bewegung (X-Richtung)
        float torque = -rb.linearVelocity.x * spinSpeed;

        // Wendet die Drehkraft an
        rb.AddTorque(torque);
    }

    void Update()
    {
        // Wenn der Ball sich bewegt
        if (rb.linearVelocity.magnitude > 0.1f)
        {
            // Dreh den Ball sichtbar
            transform.Rotate(0, 0, -rb.linearVelocity.x * spinSpeed * Time.deltaTime);
        }
    }

    void OnCollisionEnter2D(Collision2D collision)
    {
        // Wenn der Ball mit der Latte kollidiert
        if (collision.gameObject.CompareTag("Latte"))
        {
            // Latte-Sound abspielen, wenn SoundFX aktiviert sind
            if (PlayerPrefs.GetInt("soundfx", 1) == 1 && latteAudio != null)
            {
                latteAudio.Play(); // Abspielen des Latte-Sounds
            }
        }
    }
}

```

EnemyAI

```
using UnityEngine;

public class EnemyAI : MonoBehaviour
{
    public Transform ball;           // Der Ball
    public float moveSpeed;         // Bewegungsgeschwindigkeit
    public float jumpForce;         // Sprungkraft
    public float jumpThresholdY;    // Höhe, bei der der Gegner springen soll

    private Rigidbody2D rb;
    private bool isGrounded = true;   // Nur springen, wenn der Gegner den Boden berührt

    void Start()
    {
        rb = GetComponent<Rigidbody2D>(); // Holt das Rigidbody des Gegners
    }

    void Update()
    {
        if (ball != null)
        {
            // Bewege den Gegner in Richtung des Balls
            Vector2 target = new Vector2(ball.position.x, transform.position.y);
            transform.position = Vector2.MoveTowards(transform.position, target, moveSpeed *
Time.deltaTime);

            // Füge kleine Fehler bei der Bewegung hinzu, um die KI realistischer zu machen
            if (Random.value < 0.1f) // 10% Fehlerchance
            {
                transform.position = new Vector2(transform.position.x + Random.Range(-0.1f, 0.1f),
transform.position.y);
            }

            // Springen, wenn der Ball hoch genug ist
            if (ball.position.y > jumpThresholdY && isGrounded && transform.position.y + 0.2f <
ball.position.y)
            {
                rb.AddForce(Vector2.up * jumpForce, ForceMode2D.Impulse);
                isGrounded = false;
            }
        }
    }

    // Wenn der Gegner den Boden berührt, kann er wieder springen
    void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Ground"))
        {
            isGrounded = true;
        }
    }
}
```

EnemyKick

```
using UnityEngine;

public class EnemyKick : MonoBehaviour
{
    public float kickForce;           // Wie stark Ronaldo den Ball treten soll
    [Range(0,1)] public float offSpinRange; // Wie stark Ronaldo den Ball beim Treten abweichen soll
    private Rigidbody2D ballRb;        // Rigidbody vom Ball
    public AudioSource kickSound;    // Soundeffekt für den Gegner

    void OnCollisionEnter2D(Collision2D collision)
    {
        // Wenn Ronaldo den Ball berührt
        if (collision.gameObject.CompareTag("Ball"))
        {
            // Prüfen, ob dieses Objekt noch aktiv ist
            if (this == null || !gameObject.activeInHierarchy)
                return;

            // Hol das Rigidbody vom Ball
            ballRb = collision.gameObject.GetComponent<Rigidbody2D>();

            if (ballRb != null)
            {
                // Berechne die Richtung vom Ronaldo zum Ball
                Vector2 direction = (ballRb.position - (Vector2)transform.position).normalized;

                var offSpin = new Vector2(Random.Range(-this.offSpinRange, this.offSpinRange),
                    Random.Range(-this.offSpinRange, this.offSpinRange));

                // Tritt den Ball mit Kraft
                ballRb.AddForce((direction * kickForce) + offSpin, ForceMode2D.Impulse);

                // Starte die Kick-Animation
                Animator animator = GetComponent<Animator>();
                if (animator != null)
                {
                    animator.SetTrigger("KickTrigger");
                }

                // Sound nur abspielen, wenn kickSound noch existiert (abspielen, nur wenn aktiv)
                if (PlayerPrefs.GetInt("soundfx", 1) == 1 && kickSound != null)
                    kickSound.Play();
            }
        }
    }
}
```

ScoreManager

```
using UnityEngine;
using UnityEngine.UI;

public class ScoreManager : MonoBehaviour
{
    public int scorePlayer = 0;    // Punkte für Messi
    public int scoreEnemy = 0;    // Punkte für Ronaldo

    public Text scoreText;        // Punktanzeige
    public Text timerText;        // Timeranzeige
    public GameObject goalText;   // Textfeld für "GOAL!" oder Ergebnis

    public Transform ballTransform; // Ballobjekt
    public Rigidbody2D ballRb;     // Rigidbody vom Ball
    private Vector2 ballStartPosition; // Startposition Ball

    public float timeLeft = 60f;   // Zeitlimit
    private bool gameActive = true;  // Spiel läuft oder nicht

    private bool extraTime = false;  // Sind wir in Extra Time?
    private bool goldenGoalScored = false; // Wurde schon ein Golden Goal gemacht?

    public NextLevelManager nextLevelManager; // Verknüpfung zu NextLevelManager
    public AudioSource messiGoalAudio; // Drag & Drop im Inspector
    public AudioSource enemyGoalAudio; // Gegner-Tor-Sound

    void Start()
    {
        // Startposition merken
        ballStartPosition = ballTransform.position;

        // Punktestand anzeigen
        UpdateScore();
    }

    void Update()
    {
        if (gameActive)
        {
            timeLeft -= Time.deltaTime;

            if (timerText != null)
                timerText.text = Mathf.Ceil(timeLeft).ToString();
        }
    }
}
```

```

        // Zeit abgelaufen → prüfen
        if (timeLeft <= 0f)
        {
            EndGame();
        }
    }
}

void UpdateScore()
{
    // Punktestand im UI anzeigen
    scoreText.text = scorePlayer + "      " + scoreEnemy;
}

public void AddPointToPlayer()
{
    scorePlayer++;
    UpdateScore();

    // HIER wird der Sound abgespielt (Spielt nur ab, wenn aktiv)
    if (PlayerPrefs.GetInt("soundfx", 1) == 1 && messiGoalAudio != null)
        messiGoalAudio.Play();

    if (extraTime && !goldenGoalScored)
    {
        goldenGoalScored = true;
        gameActive = false;

        if (nextLevelManager != null)
        {
            nextLevelManager.ShowNextLevel(true); // true für Sieg
        }

        return;
    }

    ResetBall();
}

public void AddPointToEnemy()
{
    scoreEnemy++;
    UpdateScore();

    // Gegner-Tor-Sound abspielen (Spielt nur ab, wenn aktiv)
    if (PlayerPrefs.GetInt("soundfx", 1) == 1 && enemyGoalAudio != null)
        enemyGoalAudio.Play();
}

```



```

if (extraTime && !goldenGoalScored)
{
    goldenGoalScored = true;
    gameActive = false;

    // Nach der Niederlage wird der NextLevelManager aufgerufen und zeigt das LostPanel
    if (nextLevelManager != null)
    {
        nextLevelManager.ShowNextLevel(false); // false für Niederlage
    }

    return;
}

ResetBall();
}

void ResetBall()
{
    if (goalText != null)
    {
        goalText.SetActive(true); // "GOAL!" anzeigen
        goalText.GetComponent<Text>().text = "GOAL!";
        Invoke(nameof(HideGoalText), 0.5f);
    }

    // Ball stoppen und in Mitte setzen
    ballRb.linearVelocity = Vector2.zero;
    ballRb.angularVelocity = 0f;
    ballTransform.position = ballStartPosition;

    // Ball für 1 Sekunde einfrieren
    ballRb.bodyType = RigidbodyType2D.Kinematic;
    Invoke(nameof(ActivateBall), 1f);
}

void ActivateBall()
{
    // Ball wieder beweglich machen
    ballRb.bodyType = RigidbodyType2D.Dynamic;
}

void HideGoalText()
{
    // Text wieder ausblenden
    goalText.SetActive(false);
}

```

```

void EndGame()
{
    if (scorePlayer > scoreEnemy)
    {
        // Messi gewinnt
        if (nextLevelManager != null)
        {
            nextLevelManager.ShowNextLevel(true); // true für Sieg
        }
        gameActive = false;
    }
    else if (scoreEnemy > scorePlayer)
    {
        // Ronaldo gewinnt
        if (nextLevelManager != null)
        {
            nextLevelManager.ShowNextLevel(false); // false für Niederlage
        }
        gameActive = false;
    }
    else
    {
        // Unentschieden → Extra Time aktivieren
        extraTime = true;
        timeLeft = 60f; // Verlängerung starten (Timer quasi stoppen)
        goldenGoalScored = false;
        gameActive = true; // Spiel läuft weiter

        // Text zeigen
        if (goalText != null)
        {
            goalText.SetActive(true);
            goalText.GetComponent<Text>().text = "Golden Goal!";
        }

        // Ball anhalten
        ballRb.linearVelocity = Vector2.zero;
        ballRb.angularVelocity = 0f;
        ballRb.bodyType = RigidbodyType2D.Kinematic;
        Invoke(nameof(ActivateBall), 1f);
    }
}

public void ResetGame()
{
    // Punktestand zurücksetzen
    scorePlayer = 0;
    scoreEnemy = 0;
}

```

// Spielzeit zurücksetzen

timeLeft = 60f;

gameActive = true;

// Sonderstatus zurücksetzen

extraTime = false;

goldenGoalScored = false;

// Punktanzeige aktualisieren

UpdateScore();

// Timer-Anzeige aktualisieren

if (timerText != null)

timerText.text = Mathf.Ceil(timeLeft).ToString();

// Ball zurücksetzen

ballRb.linearVelocity = Vector2.zero;

ballRb.angularVelocity = 0f;

ballTransform.position = ballStartPosition;

ball

MovingObstacle

```
using UnityEngine;

public class MovingObstacle : MonoBehaviour
{
    public float speed = 2f;    // Geschwindigkeit der Bewegung
    public float range = 3f;    // Wie weit es sich links/rechts bewegt

    private Vector3 startPosition;

    void Start()
    {
        startPosition = transform.position; // Merkt sich die Startposition
    }

    void Update()
    {
        float movement = Mathf.Sin(Time.time * speed) * range;
        transform.position = startPosition + new Vector3(movement, 0f, 0f);
    }
}
```

SceneLoader

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class SceneLoader : MonoBehaviour
{
    public static string lastScene = "";
    public void LoadSettingsFromMainMenu()
    {
        lastScene = "MainMenu"; // direkt setzen
        SceneManager.LoadScene("SettingsScene");
    }

    public void ReturnFromSettings()
    {
        if (!string.IsNullOrEmpty(lastScene))
        {
            SceneManager.LoadScene(lastScene);
        }
    }
}
```

SoundToggle

```
using UnityEngine;
using UnityEngine.UI;

public class SoundToggle : MonoBehaviour
{
    public Sprite soundOnIcon;
    public Sprite soundOffIcon;
    public Image buttonImage;

    private bool isMuted = false;

    void Start()
    {
        // Ton-Zustand aus vorheriger Szene laden
        isMuted = PlayerPrefs.GetInt("muted", 0) == 1;
        ApplySoundState();
    }

    public void ToggleSound()
    {
        isMuted = !isMuted;

        // Zustand speichern für nächste Szene
        PlayerPrefs.SetInt("muted", isMuted ? 1 : 0);

        ApplySoundState();
    }

    private void ApplySoundState()
    {
        // Lautstärke umschalten
        AudioListener.volume = isMuted ? 0f : 1f;

        // Nur wenn Bild vorhanden → Icon wechseln
        if (buttonImage != null)
        {
            buttonImage.sprite = isMuted ? soundOffIcon : soundOnIcon;
        }

        Debug.Log("Sound " + (isMuted ? "ausgeschaltet" : "eingeschaltet"));
    }
}
```

PauseMenu

```
using UnityEngine;
using UnityEngine.UI;

public class PauseMenu : MonoBehaviour
{
    [Header("Panels")]
    public GameObject settingsPanel; // Das komplette Einstellungsmenü (UI-Panel)
    public GameObject controlsPanel; // Das Panel, das die Steuerung erklärt

    [Header("Audio")]
    public Slider volumeSlider; // Lautstärke-Slider
    public Toggle soundFXToggle; // Soundeffekte an/aus
    public AudioSource[] allAudioSources; // Alle Audioquellen im Spiel
    void Start()
    {
        // Gespeicherte Lautstärke laden
        float savedVolume = PlayerPrefs.GetFloat("volume", 1f);
        volumeSlider.value = savedVolume;

        // Lautstärke auf alle AudioSources anwenden
        foreach (AudioSource source in allAudioSources)
        {
            if (source != null)
                source.volume = savedVolume;
        }

        // SoundFX Zustand laden
        bool soundFX = PlayerPrefs.GetInt("soundfx", 1) == 1;
        soundFXToggle.isOn = soundFX;
    }

    // Öffnet das Pause- bzw. Einstellungsmenü und stoppt das Spiel
    public void OpenSettings()
    {
        settingsPanel.SetActive(true); // Menü anzeigen
        Time.timeScale = 0f; // Spiel pausieren
    }

    // Schließt das Menü und setzt das Spiel fort
    public void CloseSettings()
    {
        settingsPanel.SetActive(false); // Menü ausblenden
        controlsPanel.SetActive(false); // Falls Steuerung offen war, auch schließen
        Time.timeScale = 1f; // Spiel weiterlaufen lassen
    }

    // Wird vom Lautstärke-Slider aufgerufen
    public void SetVolume(float value)
    {
        AudioListener.volume = value; // Globale Lautstärke
    }
}
```

```

    foreach (AudioSource source in allAudioSources)
    {
        if (source != null)
            source.volume = value; // Lokale Quellen auch anpassen
    }

    PlayerPrefs.SetFloat("volume", value);
}

// Wird vom SoundFX-Toggle aufgerufen
public void ToggleSoundFX(bool enabled)
{
    PlayerPrefs.SetInt("soundfx", enabled ? 1 : 0); // 1 = an, 0 = aus
}

// Steuerung anzeigen
public void ShowControls()
{
    controlsPanel.SetActive(true);
}

// Steuerung ausblenden
public void HideControls()
{
    controlsPanel.SetActive(false);
}
}

```


GoalTrigger

```
using UnityEngine;

public class GoalTrigger : MonoBehaviour
{
    public bool isLeftGoal; // true = linkes Tor (Punkt für Ronaldo), false = rechtes Tor (Punkt für Messi)

    void OnTriggerEnter2D(Collider2D collision)
    {
        // Wenn der Ball ins Tor kommt
        if (collision.CompareTag("Ball"))
        {
            // Holt das Punktesystem (neue Methode ohne Warnung)
            ScoreManager scoreManager = FindFirstObjectByType<ScoreManager>();

            if (scoreManager != null)
            {
                // Prüft, welches Tor getroffen wurde
                if (isLeftGoal)
                    scoreManager.AddPointToEnemy(); // Punkt für Ronaldo
                else
                    scoreManager.AddPointToPlayer(); // Punkt für Messi
            }
        }
    }
}
```

MainMenu

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class MainMenu : MonoBehaviour
{
    public void StartGame()
    {
        // Spielszene laden
        SceneManager.LoadScene("Spielszene1");
    }

    public void QuitGame()
    {
        // Anwendung beenden (funktioniert nur im Build)
        Application.Quit();
        Debug.Log("Spiel wurde beendet");
    }
}
```

NextLevelManager

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class NextLevelManager : MonoBehaviour
{
    [Header("UI Panels")]
    public GameObject winPanel;    // Wird nach einem Sieg angezeigt
    public GameObject lostPanel;   // Wird nach einer Niederlage angezeigt

    [Header("Gegner im Spiel")]
    public GameObject currentEnemy;    // Aktuell aktiver Gegner
    public GameObject ronaldinhoInScene; // Ronaldinho
    public GameObject beckhamInScene;   // Beckham
    public GameObject skyBlock;         // SkyBlock einfügen bei Level 3

    [Header("Spielmanager")]
    public ScoreManager scoreManager;    // Verbindung zum ScoreManager, um das Spiel
    zurückzusetzen

    private int currentLevel = 1; // Start bei Level 1 (Ronaldo)

    void Start()
    {
        // SkyBlock am Anfang ausblenden
        if (skyBlock != null)
            skyBlock.SetActive(false);
    }

    // Wird aufgerufen, wenn das Spiel gewonnen oder verloren wurde
    public void ShowNextLevel(bool isVictory)
    {
        if (isVictory)
        {
            winPanel.SetActive(true); // WinPanel anzeigen
            lostPanel.SetActive(false); // LostPanel verstecken
            Time.timeScale = 0f;      // Spiel pausieren
        }
        else
        {
            winPanel.SetActive(false); // WinPanel verstecken
            lostPanel.SetActive(true); // LostPanel anzeigen
            Time.timeScale = 0f;      // Spiel pausieren
        }
    }

    // Startet das nächste Level, je nachdem, wer der nächste Gegner ist
    public void StartNextLevel()
    {
        // Aktuellen Gegner entfernen
        if (currentEnemy != null)
```

```

{
    Destroy(currentEnemy);
    currentEnemy = null;
}

// Nächstes Level aktivieren
if (currentLevel == 1)
{
    // Von Ronaldo zu Ronaldinho
    ronaldinhoInScene.SetActive(true);
    currentEnemy = ronaldinhoInScene;
    currentLevel = 2;
}
else if (currentLevel == 2)
{
    // Von Ronaldinho zu Beckham
    beckhamInScene.SetActive(true);
    currentEnemy = beckhamInScene;
    currentLevel = 3;

    // SkyBlock nur in Level 3 aktivieren
    if (skyBlock != null)
        skyBlock.SetActive(true);
}
else
{
    // Level 3 gewonnen → zur Siegerszene
    LoadVictoryScene();
    return;
}

// Spiel zurücksetzen
if (scoreManager != null)
    scoreManager.ResetGame();

// WinPanel ausblenden
HideMenu();

Time.timeScale = 1f; // Spiel fortsetzen
}

// Level neustarten – Szene wird einfach neu geladen
public void RetryLevel()
{
    UnityEngine.SceneManagement.SceneManager.LoadScene(
        UnityEngine.SceneManagement.SceneManager.GetActiveScene().name
    );

    HideMenu();
    Time.timeScale = 1f; // Spiel fortsetzen
}

// Spiel beenden

```

```
public void QuitGame()
{
    Application.Quit();
    Debug.Log("Spiel beendet");
}

// Versteckt beide Panels (Win & Lose)
private void HideMenu()
{
    if (winPanel != null) winPanel.SetActive(false);
    if (lostPanel != null) lostPanel.SetActive(false);
}

// Letzter Level abgeschlossen → Szene mit Gratulation laden
public void LoadVictoryScene()
{
    UnityEngine.SceneManagement.SceneManager.LoadScene("VictoryScene");
}
}
```

SettingsManager

```
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.SceneManagement;

public class SettingsManager : MonoBehaviour
{
    // Referenz zum Lautstärkeregler (UI-Slider)
    [Header("Audio")]
    public Slider volumeSlider;

    // Umschalter für Soundeffekte (an/aus)
    public Toggle soundFXToggle;

    // Panel, das die Steuerung erklärt
    [Header("UI Panels")]
    public GameObject controlsPanel;

    // Wird beim Start der Szene einmal ausgeführt
    void Start()
    {
        // Lade gespeicherte Lautstärke oder setze Standardwert 1 (max), wenn nichts gespeichert
        float savedVolume = PlayerPrefs.GetFloat("volume", 1f);
        volumeSlider.value = savedVolume;

        // Lade gespeicherten Zustand für Soundeffekte (1 = aktiv, 0 = aus)
        bool soundFX = PlayerPrefs.GetInt("soundfx", 1) == 1;
        soundFXToggle.isOn = soundFX;

        // Lautstärke sofort anwenden
        ApplyVolume();
    }

    // Wird aufgerufen, wenn der Lautstärke-Slider verändert wird
    public void ApplyVolume()
    {
        AudioListener.volume = volumeSlider.value; // Setzt globale Lautstärke
        PlayerPrefs.SetFloat("volume", volumeSlider.value); // Speichert die Einstellung
    }

    // Wird aufgerufen, wenn der SoundFX-Toggle verändert wird
    public void ToggleSoundFX()
    {
        bool isOn = soundFXToggle.isOn;
        PlayerPrefs.SetInt("soundfx", isOn ? 1 : 0); // 1 = an, 0 = a
    }

    // Steuerung anzeigen
    public void ShowControls()
    {
        controlsPanel.SetActive(true);
    }
}
```

```
}  
  
// Steuerung ausblenden  
public void HideControls()  
{  
    controlsPanel.SetActive(false);  
}  
  
}
```

Erklärungen

PlayerMovement.cs

steuert die Bewegung des Spielers (Messi). Der Spieler bewegt sich nach links und rechts mit den Pfeiltasten oder den Tasten A/D und kann mit der W-Taste oder der Pfeiltaste nach oben springen. Es wird überprüft, ob der Spieler den Boden berührt, bevor er springen kann.

PlayerKick.cs

steuert das Kicken des Balls durch den Spieler. Wenn die Leertaste gedrückt wird und der Ball in Reichweite ist, tritt der Spieler den Ball mit einer bestimmten Kraft. Es wird eine Kick-Animation abgespielt und ein Soundeffekt aktiviert.

BallSpin.cs

sorgt dafür, dass der Ball sich realistisch dreht, wenn er sich bewegt. Es berechnet die Drehkraft basierend auf der Geschwindigkeit des Balls und wendet diese an. Wenn der Ball die Latte berührt, wird ein Soundeffekt abgespielt.

EnemyAI.cs

steuert das Verhalten des Gegners. Der Gegner bewegt sich in Richtung des Balls und springt, wenn der Ball hoch genug ist. Wenn der Ball nah am Tor ist, blockiert der Gegner das Tor, ansonsten folgt er dem Ball.

EnemyKick.cs

steuert das Kicken des Balls durch den Gegner. Der Gegner tritt den Ball mit einer bestimmten Kraft und zufälligen Abweichungen. Eine Kick-Animation wird abgespielt, und ein Soundeffekt wird beim Tritt aktiviert.

ScoreManager.cs

verwaltet den Punktestand und die Spielzeit. Wenn der Ball ins Tor geht, wird ein Punkt für den Spieler oder Gegner vergeben. Es zeigt den aktuellen Punktestand und den Timer an. Wenn es unentschieden steht, wird das „Golden Goal“-Feature aktiviert.

NextLevelManager.cs

steuert den Wechsel zwischen den Levels. Es entfernt den aktuellen Gegner, fügt den nächsten hinzu und wechselt nach dem dritten Level zur Siegerszene. Es zeigt das Win- oder Lost-Panel abhängig vom Spielausgang an.

SettingsManager.cs

verwaltet die Einstellungen für Lautstärke und Soundeffekte. Der Spieler kann die Lautstärke über einen Slider anpassen und Soundeffekte ein- oder ausschalten. Es zeigt und versteckt das Steuerungs-Panel, wenn es angefordert wird.

PauseMenu.cs

ermöglicht es dem Spieler, das Spiel zu pausieren und fortzusetzen. Das Einstellungsmenü kann geöffnet werden, um die Lautstärke und Soundeffekte anzupassen. Das Menü wird geschlossen und das Spiel wird fortgesetzt, wenn das Menü geschlossen wird.

MovingObstacle.cs

steuert ein bewegliches Hindernis (SkyBlock) im Spiel. Es bewegt sich in einer festen Range hin und her, basierend auf einer Sinusfunktion. Der SkyBlock wird im dritten Level eingeführt, um das Spiel schwieriger zu machen.

SceneLoader.cs

lädt die Einstellungs-Szene und speichert die letzte Szene, um nach den Einstellungen zurückzukehren. Es ermöglicht dem Spieler, von der „SettingsScene“ zur letzten Szene zurückzukehren.

SoundToggle.cs

steuert das Ein- und Ausschalten des Sounds im Spiel. Wenn der Sound deaktiviert ist, wird er auf 0 gesetzt, andernfalls wird er auf 1 gesetzt. Der Zustand des Sounds wird in den PlayerPrefs gespeichert, sodass er beim nächsten Spielstart beibehalten wird.

GoalTrigger.cs

verwaltet das Torergebnis. Wenn der Ball ins Tor geht, wird ein Punkt für den Spieler oder den Gegner vergeben. Es prüft, welches Tor getroffen wurde und aktualisiert den Punktestand entsprechend.

MainMenu.cs

zeigt das Hauptmenü des Spiels an. Der Spieler kann das Spiel starten oder das Spiel beenden. Wenn der Spieler das Spiel startet, wird die Spielszene geladen. Wenn er das Spiel beenden möchte, wird die Anwendung geschlossen.

VictoryScene.cs

zeigt die Siegerszene nach dem letzten Level. Es spielt Animationen und zeigt ein Feuerwerk. Der Spieler kann entweder das Spiel beenden oder zurück zum Hauptmenü gehen.